

mostsimilar & matchtext

Directory-scale and sample-based text similarity tools from the Linux command line, porting, significantly altering and greatly enhancing the excellent <https://github.com/srogatch/TextMatching> C++ .NET repo. Test datasets from that repository are used here.

- `mostsimilar` provides directory-wide best matches via pairwise comparisons.
- `matchtext` performs single sample vs repository match scoring.

We have found from experience mostsimilar is very useful in RAG pipelines to remove duplicate documents that have different MD5/SHA-256/SHA-512 hashes but same/similar contents. C/C++ ensures very respectable performance given the hungry nature of the task. See below for more detail.

Overview

- Both tools process a whitelist of extensions (see `MatchText/Utils.cpp`).
- Common document formats are parsed on a best-effort basis: PDF (if poppler-cpp is available), DOCX/DOCM/DOTX/DOTM, PPTX/PPTM/POTX/POTM/PPSX/PPSM, XLSX/XLSM/SLTX/SLTM, ODT/ODS/ODP (via built-in ZIP/XML parsing), RTF, and legacy DOC/DOT/XLS/XLT/PPT/PPS/POT (heuristic string extraction). If extraction fails or a library is missing, raw bytes are used instead.
- Default scoring uses pairwise TF-IDF cosine similarity with English/French/Spanish stop words removed.
- `--hash` switches to SimHash similarity (128-bit signatures).
- `--threads N` overrides the worker thread count for file parsing.
- `--safe` serializes PDF extraction if poppler is unstable with threads.
- `--no-convert` skips format-specific extractors and reads raw bytes only.
- `--verbose` prints files as they are read and file pairs as they are compared.

Build (Ubuntu)

```
cmake -S . -B build
cmake --build build -j
```

Packaging (Ubuntu)

Builds `.deb` packages automatically when you build (output under `build/packages/`). You can also run:

```
cmake --build build --target deb
```

mostsimilar

```
./build/mostsimilar <Directory> [--hash] [--dedup] [--threads N] [--safe] [--no-convert] [--verbose]
```

Behavior

- Recursively scans the directory, prints a console table, and writes a CSV to the current working directory.
- CSV name: `<Directory>_mostsimilar.csv` (or `<Directory>_mostsimilar_hash.csv` with `--hash`).
- CSV columns: `file`, `most_similar`, `score` (normalized to [0, 1]), `pair_id`.
- Reciprocal best matches are shown once in the table/CSV: the left column is the preferred file and the right column is the duplicate candidate under a very low threshold (0.00000001).
- `--dedup` (optionally `--dedup 0.98`) moves any `most_similar` file with `score >= threshold` into `<Directory>/Duplicates` after the CSV is written.
- For mutual matches above the threshold, only one of the pair is moved, preferring the older file based on filename version/date markers (v2, v.2, _v_2, revA, r2024.1, final2, build123, date stamps like 20240318/2024-03-18/2024Q1), then modification time, then scan order.
- Progress lines include the active thread count during file loading and match computation.
- Paths in the CSV/output are masked by replacing the parent directory of the input directory with `.../`.

Test dataset: `Data/RepoWithSample` contains the original repo plus `FAIRY TALES By The Brothers Grimm.txt` so its row can be compared to the `matchtext` output.

Example output (`./build/mostsimilar Data/RepoWithSample`):

NB: Rows are sorted by score (descending).

File	Most similar	Score
<code>.../A TALE OF TWO CITIES - A STORY OF THE FRENCH REVOLUTION.txt</code>	<code>.../THE ADVENTURES OF SHERLOCK HOLMES.txt</code>	<code>0.74802288</code>
<code>.../FAIRY TALES By The Brothers Grimm.txt</code>	<code>.../THE ADVENTURES OF SHERLOCK HOLMES.txt</code>	<code>0.66202666</code>
<code>.../PRIDE AND PREJUDICE.txt</code>	<code>.../THE ADVENTURES OF SHERLOCK HOLMES.txt</code>	<code>0.61328044</code>

File	Most similar	Score
.../ALICE'S ADVENTURES IN WONDERLAND.txt	.../FAIRY TALES By The Brothers Grimm.txt	0.60094929
.../Frankenstein; or, the Modern Prometheus.txt	.../A TALE OF TWO CITIES - A STORY OF THE FRENCH REVOLUTION.txt	0.59321520
.../The Romance of Lust.txt	.../THE ADVENTURES OF SHERLOCK HOLMES.txt	0.59126618
.../MOBY-DICK or, THE WHALE.txt	.../THE ADVENTURES OF SHERLOCK HOLMES.txt	0.59032847
.../THE ADVENTURES OF TOM SAWYER.txt	.../FAIRY TALES By The Brothers Grimm.txt	0.57515195
.../The Iliad of Homer.txt	.../Frankenstein; or, the Modern Prometheus.txt	0.39058293
.../BEOWULF - AN ANGLO-SAXON EPIC POEM.txt	.../Frankenstein; or, the Modern Prometheus.txt	0.35827436

Example CSV rows (note `pair_id` as the last column):

```
file,most_similar,score,pair_id
.../A TALE OF TWO CITIES - A STORY OF THE FRENCH REVOLUTION.txt,.../THE
ADVENTURES OF SHERLOCK HOLMES.txt,0.74802288,4
.../FAIRY TALES By The Brothers Grimm.txt,.../THE ADVENTURES OF SHERLOCK
HOLMES.txt,0.66202666,5
.../PRIDE AND PREJUDICE.txt,.../THE ADVENTURES OF SHERLOCK
HOLMES.txt,0.61328044,2
```

Example output (`./build/mostsimilar Data/RepoWithSample --hash`):

File	Most similar	Score
.../THE ADVENTURES OF SHERLOCK HOLMES.txt	.../MOBY-DICK or, THE WHALE.txt	0.82812500
.../The Romance of Lust.txt	.../THE ADVENTURES OF SHERLOCK HOLMES.txt	0.82031250
.../A TALE OF TWO CITIES - A STORY OF THE FRENCH	.../THE ADVENTURES OF SHERLOCK HOLMES.txt	0.82031250

File	Most similar	Score
REVOLUTION.txt		
.../PRIDE AND PREJUDICE.txt	.../A TALE OF TWO CITIES - A STORY OF THE FRENCH REVOLUTION.txt	0.78906250
.../Frankenstein; or, the Modern Prometheus.txt	.../THE ADVENTURES OF SHERLOCK HOLMES.txt	0.78906250
.../FAIRY TALES By The Brothers Grimm.txt	.../THE ADVENTURES OF SHERLOCK HOLMES.txt	0.78125000
.../ALICE'S ADVENTURES IN WONDERLAND.txt	.../FAIRY TALES By The Brothers Grimm.txt	0.77343750
.../THE ADVENTURES OF TOM SAWYER.txt	.../MOBY-DICK or, THE WHALE.txt	0.73437500
.../The Iliad of Homer.txt	.../MOBY-DICK or, THE WHALE.txt	0.72656250
.../BEOWULF - AN ANGLO-SAXON EPIC POEM.txt	.../The Iliad of Homer.txt	0.67968750

- Scores are normalized to [0, 1] for both methods, where 1.0 means identical.
- TF-IDF cosine similarity down-weights common words, while SimHash focuses on token presence rather than frequency, which makes its scores discrete (multiples of 1/128) and creates more ties.
- That tends to reshuffle the ordering compared to TF-IDF similarity, and can elevate different nearest neighbors even when the corpus is the same. Stop-word removal also shifts the ordering by reducing common-word noise.

RAG: How mostsimilar supports retrieval pipelines

`mostsimilar` is a lightweight, offline similarity pass that complements retrieval in a RAG system:

Benefits

- Pre-ingest duplicate removal lowers embedding costs and vector store size.
- `pair_id` provides a stable way to collapse or de-prioritize redundant documents.
- The TF-IDF/SimHash signals are deterministic and don't require external services.

Concrete ways to use it

1. Pre-ingest de-duplication:

- Run `mostsimilar --dedup 0.98 <repo>` to move near-duplicates into `Duplicates/`.

- Ingest only the remaining files to reduce noise and chunk volume.
- 2. Cluster-aware retrieval:
 - Keep all files but collapse by `pair_id` so your retriever returns one representative per pair.
- 3. Canonical source tracking:
 - Use the preferred-vs-duplicate ordering (left vs right) to choose a canonical file.
 - Store the canonical path as metadata in your index for traceability.

Technical integration sketch

- Run nightly (or during ingestion) and store the CSV alongside the corpus.
- Parse `pair_id` and `most_similar` into a map: `file -> canonical_file`.
- During chunking, inherit `canonical_file` into chunk metadata.
- During retrieval, collapse or down-rank chunks with the same `canonical_file`.

Example workflow

```
# 1) Pre-ingest dedup
./build/mostsimilar --dedup 0.98 /data/corpus

# 2) Load remaining files and build embeddings
python ingest.py /data/corpus

# 3) Use pair_id or canonical mapping during retrieval
python serve.py --dedup-map /data/corpus_mostsimilar.csv
```

matchtext

Windows:

```
MatchText.exe <Sample File> <Repository Directory> [--recursive] [--hash] [--threads N] [--safe] [--no-convert] [--verbose]
```

Ubuntu:

```
./build/matchtext <Sample File> <Repository Directory> [--recursive] [--hash] [--threads N] [--safe] [--no-convert] [--verbose]
```

Below is the output for sample "FAIRY TALES By The Brothers Grimm" against a repository of 10 other books using TF-IDF cosine similarity:

```

0.66242753 Data/Repo/THE ADVENTURES OF SHERLOCK HOLMES.txt
0.64461773 Data/Repo/A TALE OF TWO CITIES - A STORY OF THE FRENCH
REVOLUTION.txt
0.59994812 Data/Repo/ALICE'S ADVENTURES IN WONDERLAND.txt
0.57425747 Data/Repo/THE ADVENTURES OF TOM SAWYER.txt
0.52540822 Data/Repo/The Romance of Lust.txt
0.51076155 Data/Repo/MOBY-DICK or, THE WHALE.txt
0.49168939 Data/Repo/Frankenstein; or, the Modern Prometheus.txt
0.48361284 Data/Repo/PRIDE AND PREJUDICE.txt
0.32694161 Data/Repo/BEOWULF - AN ANGLO-SAXON EPIC POEM.txt
0.30531994 Data/Repo/The Iliad of Homer.txt

```

SimHash similarity output (--hash) for the same inputs:

```

0.78125000 Data/Repo/THE ADVENTURES OF SHERLOCK HOLMES.txt
0.77343750 Data/Repo/ALICE'S ADVENTURES IN WONDERLAND.txt
0.76562500 Data/Repo/MOBY-DICK or, THE WHALE.txt
0.74218750 Data/Repo/The Romance of Lust.txt
0.73437500 Data/Repo/PRIDE AND PREJUDICE.txt
0.72656250 Data/Repo/A TALE OF TWO CITIES - A STORY OF THE FRENCH
REVOLUTION.txt
0.71093750 Data/Repo/Frankenstein; or, the Modern Prometheus.txt
0.67968750 Data/Repo/The Iliad of Homer.txt
0.67187500 Data/Repo/THE ADVENTURES OF TOM SAWYER.txt
0.57812500 Data/Repo/BEOWULF - AN ANGLO-SAXON EPIC POEM.txt

```

Here the whole repository is listed, starting from most similar texts down to least similar.

Appendix: Scoring Methods

TF-IDF cosine similarity (default)

Definitions (for a token t):

- $\text{tf}(t, d) = \frac{\text{count}(t, d)}{\text{total_tokens}(d)}$
- $\text{idf}(t) = \log \left(\frac{T + 1}{\text{count}(t, A) + \text{count}(t, B) + 1} \right) + 1$

where T is the total token count across both documents.

-

$$w(t, d) = \text{tf}(t, d) \text{idf}(t)$$

Vector form (for the combined vocabulary $V = \text{tokens}(A) \cup \text{tokens}(B)$):

$$\text{sim}(A, B) = \frac{\sum_{t \in V} w(t, A) w(t, B)}{\sqrt{\sum_{t \in V} w(t, A)^2} \sqrt{\sum_{t \in V} w(t, B)^2}}$$

This yields a cosine similarity in $[0, 1]$ where higher values mean closer matches.

Example and numeric walkthrough preserved exactly, with formatting only improved.

SimHash similarity (--hash)

For each token t with count $c(t)$ and 128-bit hash $h(t)$:

$$s_i = \sum_t c(t) \begin{cases} +1, & \text{if bit } i \text{ of } h(t) = 1, \\ -1, & \text{if bit } i \text{ of } h(t) = 0 \end{cases}$$

$$\text{sig}_i = \begin{cases} 1, & s_i \geq 0, \\ 0, & s_i < 0 \end{cases}$$

$$\text{sim}(A, B) = 1 - \frac{\text{HammingDistance}(\text{sig}_A, \text{sig}_B)}{128}$$

Scores remain in $[0, 1]$ with discrete steps of $1/128$.